Reviewer Pack — Minimal Hodge Index and Communication Complexity Rank Varian…

Entry 9753e218bda9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 08:14:07 UTC

Minimal Hodge Index and Communication Complexity Rank Variance Bound

Entry ID: `9753e218bda9` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-07 08:14:07 UTC

1. Conjecture

Field A (mathematical branch): Hodge Theory **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

> For every Boolean function f with input length n, the minimal Hodge index of its associated algebraic variety V_f is linearly correlated with the variance in communication complexity rank for all circuits computing f, such that h(V_f) = Ω(log^2(n)) * Var(Rank_C(f))

Rationale (proposer's reasoning):

> Hodge Theory provides a rich framework for studying algebraic varieties, which could offer new insights into the geometric properties of Boolean functions. The variance in communication complexity rank measures the diversity of circuit representations for a function, and a strong correlation with Hodge index might reveal a deep connection between geometric properties and circuit complexity.

Taxonomy category: `HODGEINDEXCOMMUNICATIONCOMPLEXITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d2b3d0cb8cd48b81`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for each Boolean function f with input length n, the ratio h(V_f) / Var(Rank_C(f)) is greater than or equal to a constant c * log^2(n) across at least 30 random seeds, where h(V_f) is the minimal Hodge index of the associated algebraic variety V_f and Var(Rank_C(f)) is the variance in communication complexity rank for all circuits computing f. The conjecture is falsified if this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Hodge Index [hep-th AND comm-complexity] - Variance in Communication Complexity Rank [math.AG AND rank variation] - [Hodge Theory] AND [Communication Complexity Rank Variance]

Top relevant hits considered: - [http://arxiv.org/abs/1304.3539v2] The arithmetic Hodge index theorem for adelic line bundles II - [http://arxiv.org/abs/1901.05780v2] Hodge filtration, minimal exponent, and local vanishing - [http://arxiv.org/abs/1801.10489v3] Hodge level for weighted complete intersections - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1901.11354v2] The monic rank - [http://arxiv.org/abs/1305.1726v2] On differences between the border rank and the smoothable rank of a polynomial - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            continue
        denom = A[i][i]
        for j in range(n):
            A[i][j] /= denom
        for j in range(n):
            if i != j:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
    rank = sum(1 for row in A if any(row))
```

```
    return rank

def hodge_index(matrix):
    n = len(matrix)
    if n == 0:
        return 0
    return gaussian_elimination(matrix)

def communication_complexity_rank(f, n):
    # Placeholder function to simulate the calculation of communication complexity rank
    # This is a dummy implementation and should be replaced with actual logic
    return random.randint(1, n)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0.0
    instances_tested = 0
    n_max = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        hodge_index_sum = 0.0
        rank_variance_sum = 0.0
        for _ in range(5):  # Generate multiple circuits per function
            f = [random.randint(0, 1) for _ in range(n)]
            hodge_index_value = hodge_index([[f[i] - f[j] for j in range(n)] for i in range(n)])
            rank = communication_complexity_rank(f, n)
            rank_variance_sum += (rank - n / 2) ** 2
            instances_tested += 1
            n_max = max(n_max, n)
            hodge_index_sum += hodge_index_value

        if instances_tested < 30:
            conjecture_holds = False
            counterexample = "insufficient_instances"
            break

        mean_rank_variance = rank_variance_sum / instances_tested
        ratio = hodge_index_sum / (instances_tested * mean_rank_variance)
        total_metric_value += ratio

    if not conjecture_holds:
        return {
            "metric_name": "h(V_f) / Var(Rank_C(f))",
            "metric_value": None,
            "instances_tested": instances_tested,
            "n_max": n_max,
            "conjecture_holds": conjecture_holds,
            "counterexample": counterexample
        }

    mean_metric_value = total_metric_value / len(n_values)
    return {
        "metric_name": "h(V_f) / Var(Rank_C(f))",
```

```
        "metric_value": mean_metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_958976f3.py", line 102, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_958976f3.py", line 61, in run_trial
    hodge_index_value = hodge_index([[f[i] - f[j] for j in range(n)] for i in range(n)])
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_958976f3.py", line 40, in hodge_index
    return gaussian_elimination(matrix)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_958976f3.py", line 22, in gaussian_elim
    A[i], A[max_row] = A[max_row], A[i]
                       ~^^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means the pre-registered support condition was not unambiguously met. | next: Re-run the test with appropriate error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14328
2	preregistration	ollama_remote	glm4:latest	0	0	10288
3	novelty	ollama_remote	glm4:latest	0	0	12743
4	novelty	ollama_remote	glm4:latest	0	0	12762
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39412
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10209
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16281
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12713
9	judge	ollama_remote	glm4:latest	0	0	29160

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 157897 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9753e218bda9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9753e218bda9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/9753e218bda9.tar.gz` (if generated)